SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission

Student Name	N.J.Sreelakshmi
Register Number	1920260072
Course Code & Name	CSA0209- C PROGRAMMING
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative (Tasks 1 to 4: header files, input/output, type conversion, compilation process)
GitHub Repository URL	https://github.com/sreelakshminj6825-ctrl/co2
Submission Date	27/10/2026

Program used throughout: a monthly billing program for a rural electricity co-operative. Meter readings are 16-bit (0 to 65535, rolling over after 65535). Tariff: first 100 units free, units 101 to 200 at Rs 2.25, units above 200 at Rs 4.50, plus a fixed charge of Rs 50 per bill. Money is held in whole **paise** in a `long`. All programs were compiled and run with gcc 13.3.0 on Ubuntu 24.04.

TASK 1 — Organise the Program with Header Files and Conditional Compilation

Marks: 3 **Bloom's Level:** K3 – Apply

Task / Problem Statement

Organise the constants and function prototypes of the billing program into a header file and add a debugging mode. Discuss the structure of a C program, include guards, conditional compilation with `#ifdef DEBUG` (switched on with `gcc -DDEBUG`), and the difference between `#include <file>` and `#include "file"`. As the alternative, compare the debugging switch `if (debug) printf(...)` controlled by a run-time variable.

Aim

To split the billing program into a header file (`tariff.h`) and source files, protect the header with include guards, add a debugging mode that costs nothing in the normal build, and compare it with a run-time debug flag in time and space.

Concept / Theory

Structure of a C program. A C source file is read from top to bottom and normally has these parts, in this order:

Part	What it contains in the billing program
Header files / preprocessor lines	`#include <stdio.h>` (system header) and `#include "tariff.h"` (our own header); `#define` constants and macros.
Global declarations	Constants (slab limits, rates, fixed charge), the `Consumer` structure type and the **function prototypes**. These go in the header so every file sees the same declarations.
`main()`	Where execution starts: it calls `compute_bill()` and `print_bill()` for sample readings.
Other functions	`compute_bill()` and `print_bill()` are defined in `tariff.c`; each has its prototype in `tariff.h`.

Include guards. `#ifndef TARIFF_H ... #define TARIFF_H ... #endif` makes the compiler read the header body only once per source file. A header is often included twice without the programmer noticing (for example `main.c` includes `a.h` and `b.h`, and both include `tariff.h`). Macros and prototypes can be repeated harmlessly, but a `typedef` or `struct` definition cannot, so without guards the compiler stops with a "conflicting types" or "redefinition" error (shown in Figure 1.3).

Conditional compilation. `#ifdef DEBUG` keeps or removes lines **before** compilation. If `DEBUG` is not defined, the debug lines are deleted by the preprocessor and do not exist in the executable. `gcc -DDEBUG` is the same as writing `#define DEBUG` at the top of every file.

`#include <file>` and `#include "file"`. Angle brackets search only the system include directories (the list is printed in Figure 1.4). Double quotes search first the directory of the current source file and, if the file is not found there, fall back to the system directories. So system headers use `<>` and the project's own headers use `""`.

Procedure / Algorithm

1. Move the constants, the `Consumer` type and the prototypes into `tariff.h` and wrap the file in an include guard.
2. Define a `DBG(...)` macro in the header: with `-DDEBUG` it expands to `printf("[DEBUG] " ...)`, otherwise to `((void)0)`.
3. Put `compute_bill()` and `print_bill()` in `tariff.c` and call `DBG(...)` at the points where intermediate values are interesting (slab 2 added, slab 3 added).
4. Write `main.c` which includes `tariff.h` and bills 0, 150 and 350 units.
5. Compile twice: `gcc main.c tariff.c` (normal) and `gcc -DDEBUG main.c tariff.c` (debug) and compare the output.

6. Test the include guard by including the header twice with and without the guard, and test <> against "".
7. Measure the alternative (a run-time `debug` variable) for time per call and for executable size.

Program / Source Code

tariff.h — constants, DEBUG macro, shared type and prototypes (with include guard)

```
#ifndef TARIFF_H
#define TARIFF_H

/* ---- constants (global, defined once) ---- */
#define SLAB1_UNITS        100
#define SLAB2_UNITS        200
#define RATE2_PAISE        225
#define RATE3_PAISE        450
#define FIXED_CHARGE_PAISE 5000L

/* ---- debugging mode: active only when compiled with -DDEBUG ---- */
#ifdef DEBUG
  #define DBG(...) printf("[DEBUG] " __VA_ARGS__)
#else
  #define DBG(...) ((void)0)
#endif

/* ---- data type shared by all files ---- */
typedef struct {
    unsigned int household_no;
    char         category;     /* D, C or I */
} Consumer;

/* ---- function prototypes ---- */
long compute_bill(int units);
void print_bill(long paise);

#endif /* TARIFF_H */
```

tariff.c — function definitions

```
#include <stdio.h>
#include "tariff.h"

long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    DBG("compute_bill: units=%d, start paise=%ld\n", units, paise);
    if (units > SLAB2_UNITS) {
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        DBG("slab3 added, paise=%ld\n", paise);
    }
    if (units > SLAB1_UNITS) {
        int in2 = (units > SLAB2_UNITS ? SLAB2_UNITS : units) - SLAB1_UNITS;
        paise += (long)in2 * RATE2_PAISE;
        DBG("slab2 added (%d units), paise=%ld\n", in2, paise);
    }
    return paise;
}

void print_bill(long paise)
{
    printf("Bill = Rs %ld.%02ld\n", paise / 100, paise % 100);
}
```

main.c — driver

```
#include <stdio.h>
#include "tariff.h"

int main(void)
{
    int tests[] = {0, 150, 350};
    for (int i = 0; i < 3; i++) {
        printf("Units = %d\n", tests[i]);
        print_bill(compute_bill(tests[i]));
    }
    return 0;
}
```

Output Screenshot

```
Task 1 - Terminal

student@csa0210:~$ gcc -Wall -Wextra main.c tariff.c -o bill && ./bill
Units = 0
Bill = Rs 50.00
Units = 150
Bill = Rs 162.50
Units = 350
Bill = Rs 950.00
student@csa0210:~$
```

Figure 1.1 — Normal build: no debug output, correct bills (Rs 50.00, Rs 162.50, Rs 950.00).

```
Task 1 - Terminal (DEBUG mode)

student@csa0210:~$ gcc -Wall -Wextra -DDEBUG main.c tariff.c -o bill_dbg && ./bill_dbg
Units = 0
[DEBUG] compute_bill: units=0, start paise=5000
Bill = Rs 50.00
Units = 150
[DEBUG] compute_bill: units=150, start paise=5000
[DEBUG] slab2 added (50 units), paise=16250
Bill = Rs 162.50
Units = 350
[DEBUG] compute_bill: units=350, start paise=5000
[DEBUG] slab3 added, paise=72500
[DEBUG] slab2 added (100 units), paise=95000
Bill = Rs 950.00
student@csa0210:~$
```

Figure 1.2 — Compiled with -DDEBUG: the same bills plus the intermediate [DEBUG] lines.

```
Task 1 - Terminal (include guards)

student@csa0210:~$ gcc -c twice2.c   # header WITHOUT guard, included twice
In file included from twice2.c:2:
noguard.h:20:3: error: conflicting types for ‘Consumer’; have ‘struct <anonymous>’
   20 | } Consumer;
      |   ^~~~~~~~
In file included from twice2.c:1:
noguard.h:20:3: note: previous declaration of ‘Consumer’ with type ‘Consumer’
student@csa0210:~$ gcc -Wall -c twice.c   # header WITH guard, included twice
compiled with no errors
student@csa0210:~$
```

Figure 1.3 — Header included twice: error without the include guard, clean compile with it.

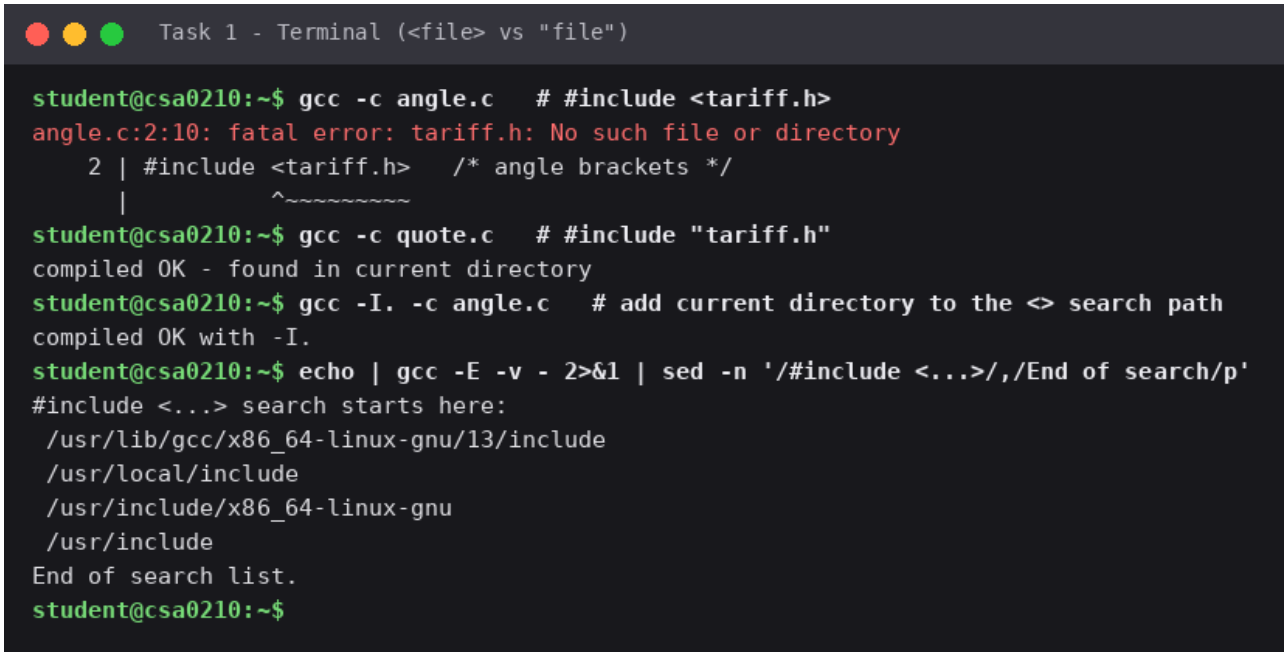


Figure 1.4 — #include <tariff.h> fails, #include "tariff.h" works, and the <> search list.

Analysis / Discussion

Debug mode. Figures 1.1 and 1.2 show that the bills are identical in both builds. In the debug build the trace for 350 units shows the bill growing in steps: 5,000 paise (fixed) → 72,500 after slab 3 (150 × 450 = 67,500 added) → 95,000 after slab 2 (100 × 225 = 22,500 added) = Rs 950.00. The executable grows from 1,975 to 2,195 bytes of code and read-only data (`size bill bill_dbg` in Figure 1.5), because the debug statements and their format strings are compiled in only for the debug build.

Include guard. In Figure 1.3, without the guard the second inclusion of `tariff.h` defines the `Consumer` type again and gcc reports `conflicting types for 'Consumer'`. With the guard the second inclusion is skipped and the file compiles. The guard is needed because of the one-definition rule for types, even though repeated macros and prototypes would have been accepted.

Angle brackets and quotes. In Figure 1.4 `#include <tariff.h>` gives `fatal error: tariff.h: No such file or directory` because the current directory is not in the system search list. `#include "tariff.h"` compiles because the quoted form looks in the current directory first. Adding `-I.` puts the current directory on the `<>` path, which is why the third command also works.

Alternative Concept and Complexity Analysis — run-time variable instead of #ifdef DEBUG

If the debug output is controlled by a variable (`int debug = 0;` and `if (debug) printf(...)`), the program is compiled **once** and debugging is switched on while it runs (for example from a command-line option), with no recompilation. The cost is that the test is executed on every call and the debug code always stays in the executable.

rt.c — the two approaches compared in a loop of 200 million calls

```
#include <stdio.h>
#include <time.h>

#define N 200000000L

static volatile long sink;
static int debug = 0;                    /* run-time debug flag */

/* Approach A: compile-time switch (DEBUG not defined -> no debug code) */
#ifdef DEBUG
static inline long calc ct(int u) { printf("dbg %d\n", u); return u * 225L + 5000; }
#else
static inline long calc ct(int u) { return u * 225L + 5000; }
#endif

/* Approach B: run-time switch - the test is executed on every call */
static inline long calc_rt(int u)
```

```
{
    if (debug) printf("dbg %d\n", u);
    return u * 225L + 5000;
}

int main(void)
{
    clock_t t;
    long s = 0;

    t = clock();
    for (long i = 0; i < N; i++) s += calc_ct((int)(i & 255));
    double a = (double)(clock() - t) / CLOCKS_PER_SEC;
    sink = s;

    s = 0;
    t = clock();
    for (long i = 0; i < N; i++) s += calc_rt((int)(i & 255));
    double b = (double)(clock() - t) / CLOCKS_PER_SEC;
    sink = s;

    printf("#ifdef DEBUG off  : %.3f s for %ld calls\n", a, N);
    printf("if (debug) flag   : %.3f s for %ld calls\n", b, N);
    return 0;
}
```

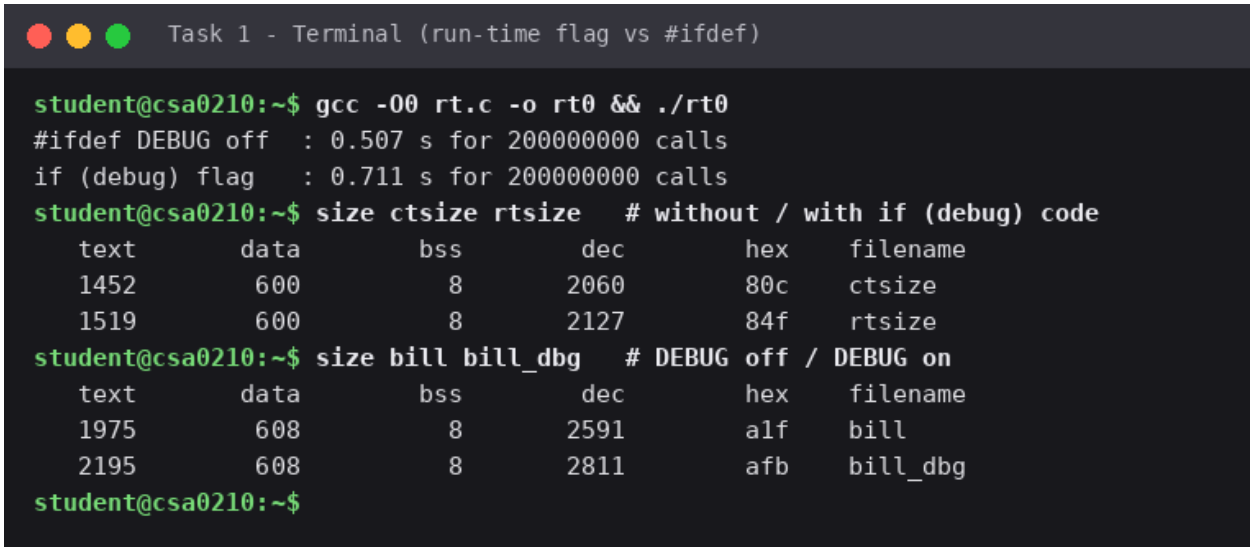


Figure 1.5 — Time per call (rt.c) and size of the executable code for each approach.

Aspect	#ifdef DEBUG (compile time)	if (debug) (run time)
Time per call	No extra instructions when DEBUG is off: 0 extra work. Measured 0.507 s for 200,000,000 calls.	One load, one compare and one branch on every call: O(1) per call, O(n) over n calls. Measured 0.711 s for the same 200,000,000 calls, about 40% slower here (about 1 ns per call). Repeated runs gave 0.62 to 0.71 s, i.e. 25% to 40% slower.
Space in executable	Debug code is removed in the normal build. `size`: 1,452 bytes of code and data.	Debug code and its strings always remain: 1,519 bytes, i.e. 67 bytes for a single debug statement; more for every extra one.
Switching on debugging	Needs recompilation with `-DDEBUG`.	Can be turned on in a running or installed program.
Risk	Debug code is absent from the released program.	Internal values can be printed in the field by mistake; the flag must be tested everywhere.

The extra time per call is a constant, so both approaches are O(1) per call, but for a billing program that is called millions of times the compile-time switch is cheaper and keeps the delivered executable smaller. The run-time flag is chosen when the program must be diagnosed on site without a compiler.

Result

The constants, shared type and prototypes were moved into a header with include guards, and a `-DDEBUG` mode was added that prints the intermediate bill values only when requested. The include guard was shown to prevent a "conflicting types" error, `<>` and `""` were shown to search different places, and the compile-time switch was shown to cost nothing at run time, against roughly 25% to 40% extra time and 67 extra bytes per debug statement for the run-time flag.

TASK 2 — Read and Print Data Correctly

Marks: 4 **Bloom's Level:** K3 – Apply

Task / Problem Statement

Design the input and output of the billing program using `scanf` and `printf`. Discuss the correct format specifier for each data type, what happens when a wrong specifier is used (shown by a test), how to check the return value of `scanf` to detect invalid input such as letters, and how to print rupees and paise from an integer amount in paise (for example Rs 162.50). As the alternative, read each line with `fgets()` and convert it with `sscanf()` or `strtol()`.

Aim

To read the household number, category and two meter readings with the correct specifiers, to reject invalid input using the return value of `scanf`, and to print a correctly formatted bill; and to compare line-based input (`fgets` + `strtol`) with direct `scanf` in time, space and recovery from bad input.

Concept / Theory

A format specifier tells `scanf`/`printf` how many bytes to use and how to interpret them. A wrong specifier is not detected by the compiler in most cases; the function reads or prints the wrong bytes (undefined behaviour). `scanf` returns the **number of items successfully stored** (or `EOF`), so comparing that value with the number expected is the standard validity check. Money is printed from integer paise with `p / 100` (rupees) and `p % 100` printed as `%02ld`, which keeps the leading zero (5 paise prints as .05).

Data type	printf specifier	scanf specifier	Used for in the program
`int`	`%d`	`%d`	units passed to compute_bill
`unsigned int`	`%u`	`%u`	household number, units consumed
`unsigned short`	`%hu`	`%hu`	meter readings (16-bit register)
`long`	`%ld`	`%ld`	money in paise
`long long`	`%lld`	`%lld`	district totals
`float`	`%f`	`%f`	not used for money
`double`	`%f` or `%lf`	`%lf` (must have l)	not used for money
`char`	`%c`	`%c` (use " %c" to skip spaces)	category D / C / I
`char[]` (string)	`%s`	`%s` (stops at space)	names

Procedure / Algorithm

1. Declare `household` (unsigned int), `category` (char), `prev` and `curr` (unsigned short).
2. Read all four items with one `scanf("%u %c %hu %hu", ...)` and store its return value in `n`.
3. If `n != 4`, print how many items were read and stop with a non-zero exit status.
4. Compute units with unsigned 16-bit subtraction `(unsigned short)(curr - prev)` so that a rollover gives the right answer.
5. Compute the bill in paise with the slab rules and print it as `Rs %ld.%02ld` using `/ 100` and `% 100`.
6. Test `spec.c` to print every type with its specifier and then with deliberately wrong specifiers.
7. Write the alternative `task2_alt.c` that reads a whole line with `fgets` and converts it with `strtol`, and compare it with a `scanf` retry loop.

Program / Source Code

task2.c — input with scanf, validation, and bill printing

```
#include <stdio.h>
#include <stdlib.h>

#define FIXED CHARGE PAISE 5000L

long compute_bill(int units)
{
    long p = FIXED CHARGE PAISE;
    if (units > 200) p += (long)(units - 200) * 450;
    if (units > 100) p += (long)((units > 200 ? 200 : units) - 100) * 225;
    return p;
}

int main(void)
{
    unsigned int  household;
    char          category;
    unsigned short prev, curr;

    printf("Enter household no, category (D/C/I), previous, current: ");
    /* scanf returns the number of items successfully read */
    int n = scanf("%u %c %hu %hu", &household, &category, &prev, &curr);
    if (n != 4) {
        printf("Invalid input: only %d of 4 items were read.\n", n);
        return 1;
    }
    unsigned int units = (unsigned short)(curr - prev);
    long paise = compute bill((int)units);

    printf("\n----------- ELECTRICITY BILL -----------\n");
    printf("Household No : %u\n", household);
    printf("Category     : %c\n", category);
    printf("Previous     : %hu\n", prev);
    printf("Current      : %hu\n", curr);
    printf("Units used   : %u\n", units);
    printf("Total payable: Rs %ld.%02ld\n", paise / 100, paise % 100);
    printf("----------------------------------------\n");
    return 0;
}
```

spec.c — format specifiers and wrong-specifier test

```
#include <stdio.h>
int main(void)
{
    int           d  = -42;
    unsigned int  u  = 4000000000u;
    long          ld = 5000000000L;
    long long     lld= 9000000000000000000LL;
    float         f  = 162.5f;
    double        lf = 162.5;
    char          c  = 'D';
    char          s[] = "Rural";

    printf("%%d  int           : %d\n", d);
    printf("%%u  unsigned      : %u\n", u);
    printf("%%ld long          : %ld\n", ld);
    printf("%%lld long long    : %lld\n", lld);
    printf("%%f  float         : %f\n", f);
    printf("%%lf double        : %lf\n", lf);
    printf("%%c  char          : %c\n", c);
    printf("%%s  string        : %s\n", s);
    printf("\n-- WRONG specifiers --\n");
    printf("%%d  for long 5000000000      : %d\n", ld);
    printf("%%d  for unsigned 4000000000  : %d\n", u);
    printf("%%c  for int 68               : %c\n", 68);
    printf("%%u  for int -42              : %u\n", d);
    return 0;
}
```

task2_alt.c — alternative: fgets() + strtol()

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <string.h>
```

```c
int main(void)
{
    char line[64];                          /* buffer: L+2 bytes needed */
    int  tries = 0;
    for (;;) {
        printf("Enter current reading (0-65535): ");
        if (!fgets(line, sizeof line, stdin)) { puts("End of input."); return 1; }
        tries++;
        char *end; errno = 0;
        long v = strtol(line, &end, 10);
        if (end == line || (*end != '\n' && *end != '\0') || errno || v < 0 || v > 65535) {
            printf("  Rejected \"%.*s\" - try again\n", (int)strcspn(line, "\n"), line);
            continue;                       /* whole line already consumed */
        }
        printf("  Accepted %ld after %d attempt(s)\n", v, tries);
        break;
    }
    return 0;
}
```

scanf_loop.c — retrying scanf after letters (shows why the buffer must be cleared)

```c
#include <stdio.h>
int main(void)
{
    int v, n, tries = 0;
    while (tries < 3) {
        printf("Enter current reading: ");
        n = scanf("%d", &v);
        tries++;
        if (n == 1) { printf("  Accepted %d\n", v); return 0; }
        if (n == EOF) { puts("  EOF"); return 1; }
        printf("  scanf returned %d (try %d): bad text still in buffer\n", n, tries);
        /* without clearing, scanf would fail again on the same letters */
    }
    puts("  gave up - letters were never consumed");
    return 1;
}
```

Output Screenshot

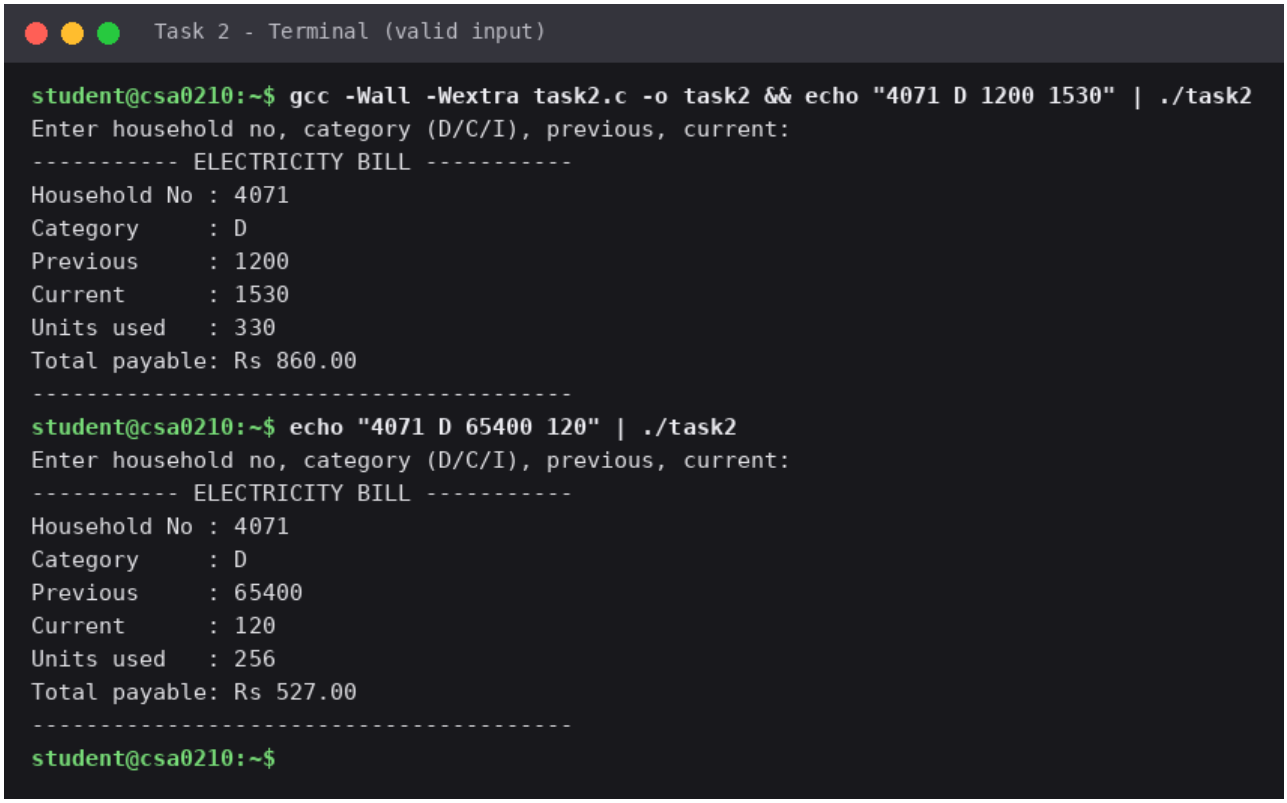


Figure 2.1 — Valid input: 330 units = Rs 860.00; and rollover case 65400 → 120: 256 units = Rs 527.00.

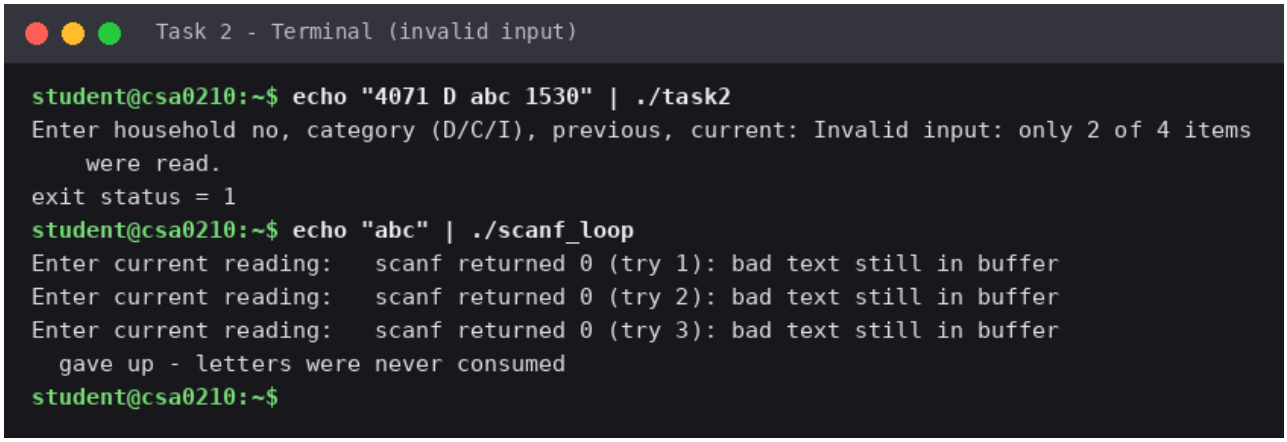


Figure 2.2 — Invalid input (letters): scanf returns 2, not 4, so the program rejects the line; retry loop never recovers.

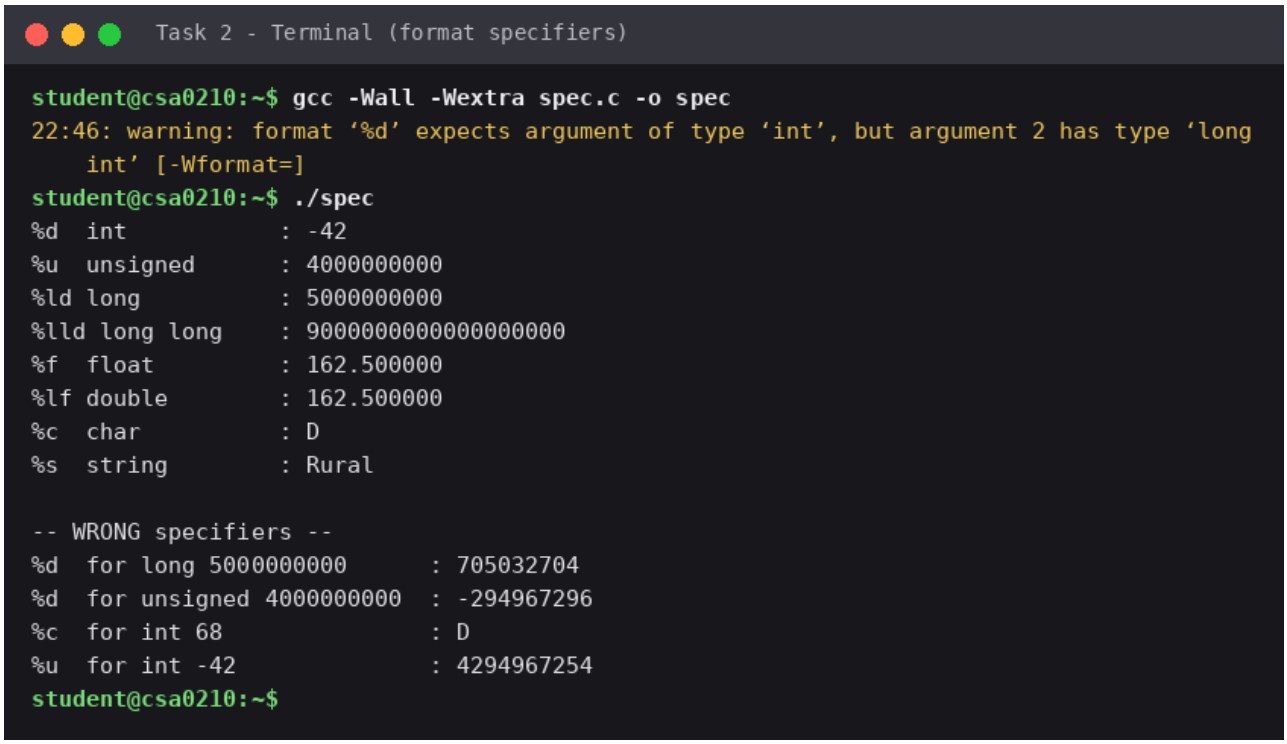


Figure 2.3 — Correct specifiers for each type, the gcc warning, and the wrong-specifier results.

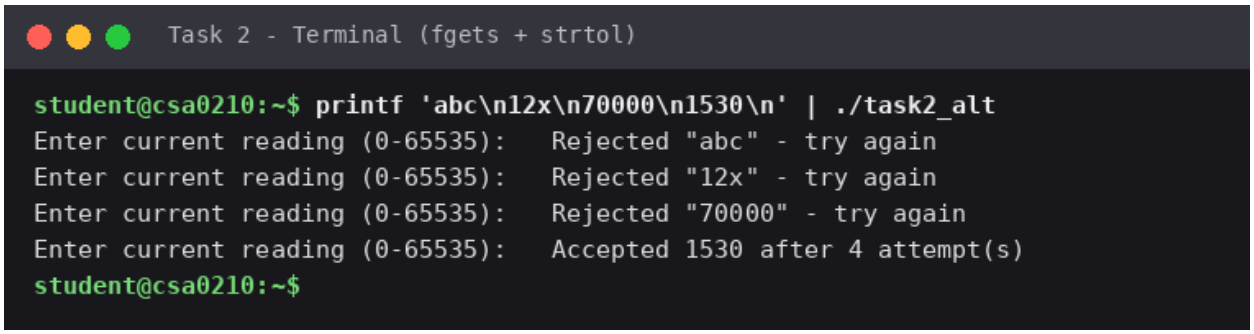


Figure 2.4 — Alternative: fgets + strtol rejects abc, 12x and 70000, then accepts 1530.

Analysis / Discussion

Correct bill. For readings 1200 and 1530 the program prints 330 units and Rs 860.00: 100 units are free, 100 × 225 = 22,500 paise, 130 × 450 = 58,500 paise, plus 5,000 paise fixed = 86,000 paise = Rs 860.00. For the rollover reading (65400 then 120) the unsigned subtraction gives 256 units, so the bill is 22,500 + 56 × 450 + 5,000 = 52,700 paise = Rs 527.00.

Wrong specifiers (Figure 2.3). `%d` with the `long` value 5,000,000,000 printed **705032704**, which is 5,000,000,000 − 2^{32} = the low 32 bits only; gcc warned `format '%d' expects argument of type 'int', but argument 2 has type 'long int'. %d` with the unsigned value 4,000,000,000 printed **−294967296** because the same bit pattern was read as a signed number, and `%u` with −42 printed **4294967254** for the opposite reason. Nothing stopped the program, so a wrong specifier silently produces a wrong bill. `-Wall -Wextra` should always be used because it catches the long/int case.

Invalid input (Figure 2.2). With the input `4071 D abc 1530`, `scanf` stored the household number and the category and then failed at the letters, so it returned 2. The program tests `n != 4`, prints `Invalid input: only 2 of 4 items were read.` and exits with status 1 instead of billing garbage. The retry-loop test shows a second problem: after a failure the letters stay in the input buffer, so calling `scanf("%d")` again fails every time (three attempts, all returning 0). The bad characters must be discarded (for example `while (getchar() != '\n');`) before retrying.

Printing rupees and paise. `printf("Rs %ld.%02ld", p / 100, p % 100)` prints 16250 paise as `Rs 162.50` and 5 paise as `Rs 0.05`. The `02` in `%02ld` pads the paise to two digits; without it 5 paise would print as `0.5`, which is wrong by a factor of ten.

Alternative Concept and Complexity Analysis — fgets() with sscanf()/strtol()

Here every input line is first read completely into a buffer with `fgets`, and then converted with `strtol` (or `sscanf`). The program of Figure 2.4 rejects `abc` (no digits), `12x` (extra characters after the number, detected through the end pointer) and `70000` (outside 0 to 65535), and accepts `1530` at the fourth attempt. Because `fgets` has already removed the whole bad line from the input stream, the next attempt starts clean, with no extra buffer-clearing code.

Aspect	Direct scanf	fgets + strtol / sscanf
Time for a line of length L	O(L): characters are scanned and converted in one pass.	O(L) to read the line plus O(L) to convert it: still O(L) overall (about two passes, so a larger constant).
Extra buffer space	None in the program (stdio's own buffer only): O(1).	A line buffer of at least L + 2 bytes (the newline and the terminating \0): O(L); 64 bytes in this program. Lines longer than the buffer need an extra check.
Recovery from invalid input	Bad characters remain in the stream; the program must discard them or it loops forever (Figure 2.2).	The whole line is already consumed; the program just asks again.
Error detail	Only a count of items read.	`strtol` gives the end pointer and `errno`, so "no digits", "trailing junk" and "out of range" can be told apart.
Fields on one line	Easy: `scanf("%u %c %hu %hu").`	Use `sscanf` on the buffer (same specifiers).

Both approaches are linear in the line length, so the choice is about robustness rather than speed: for a billing program that must not accept corrupt readings, `fgets` with `strtol` is the better design.

Result

Correct format specifiers were used for every data type, the wrong-specifier test showed the silent errors they cause (705032704 instead of 5,000,000,000), invalid input was detected from the return value of `scanf`, and the bill was printed correctly as rupees and paise (Rs 860.00 and Rs 527.00). The `fgets`/`strtol` method was shown to recover from bad input more simply, at the cost of an O(L) line buffer.

TASK 3 — Control Type Conversion and Precision

Marks: 4 **Bloom's Level:** K4 – Analyse

Task / Problem Statement

Identify every place in the billing program where values of different types are combined. Discuss implicit conversion (integer promotion and the usual arithmetic conversions), integer division (for example `units / 100`) and where it gives a wrong result, explicit casting and the effect of casting a large `long` to `int`, and the rounding rules when converting a fractional charge to paise. Show the output of at least three expressions that give an unexpected result and explain each. As the alternative, do all calculations in `float` and round only once at the end, and measure how the error in the total grows with the number of bills n.

Aim

To locate every type mix in the billing program, to demonstrate and explain surprising conversion results, to fix one consistent rounding rule, and to compare integer paise with floating-point arithmetic in error, time and space.

Concept / Theory

Integer promotion: operands of type `char` and `short` are converted to `int` before any arithmetic. **Usual arithmetic conversions:** when the two operands then still differ, the "smaller" type is converted to the "larger" one in this order: `int` → `unsigned int` → `long` → `unsigned long` → `float` → `double`. A signed value combined with an unsigned one of the same size becomes unsigned, so −10 turns into 4,294,967,286. **Integer division** truncates toward zero: `250 / 100` is 2, and `2 / 3` is 0. **Explicit casting** `(type)expr` forces a conversion; casting to a narrower type keeps only the low-order bits, and casting a floating value to an integer discards the fraction. **Rounding:** a conversion such as `(int)x` truncates; rounding to the nearest value needs `+ 0.5` or `lround()`.

No.	Where in the program	Types combined	Effect / safeguard
1	`curr - prev` (meter units)	`unsigned short` − `unsigned short`, promoted to `int`, cast back to `unsigned short`	The cast makes the result modulo 65536, which handles rollover.
2	Passing `units` to `compute_bill(int)`	`unsigned int` → `int`	Safe only while units < 2^{31}; units never exceed 65535.
3	`(long)(units - SLAB2) * RATE3`	`int` × `int` converted to `long`	The cast to `long` before multiplying prevents int overflow of large totals.
4	`paise / 100` and `paise % 100`	`long` with `int` constant	Integer division is wanted here: it separates rupees and paise.
5	`units > limit` comparisons	`int` with `unsigned int`	The signed value is converted to unsigned (gcc warns with -Wsign-compare).
6	Category tests such as `c == 'D'`	`char` promoted to `int`	Safe: both sides are int after promotion.
7	Rupees → paise (`rate * 100`)	`double` → `long`/`int`	Needs an explicit rounding rule (see below).
8	`printf` arguments	`float` → `double`, `short` → `int`	Default argument promotion: `%f` receives a double, `%d` receives an int.

Procedure / Algorithm

1. List every expression of the billing program in which operands of different types meet (table above).
2. Write `task3.c` containing expressions for implicit conversion, integer division, explicit casts and rounding, and print each result next to the expected value.

3. Compile with `-Wall -Wextra -Wconversion` to see which conversions gcc reports.
4. Choose one rounding rule and check it against truncation and `lround()`.
5. Write `task3_alt.c`: generate n bills with a fractional rate of 225.5 paise per unit, total them (a) in `float` rounding once at the end, (b) in `double` rounding once at the end and (c) in integer paise rounded per bill, and compare each with the exact total for n = 10 to 1,000,000.

Program / Source Code

task3.c — conversion points and surprises

```
#include <stdio.h>
#include <limits.h>
#include <math.h>

int main(void)
{
    /* ---- 1. implicit conversion / promotion ---- */
    unsigned int  u = 5;
    int           i = -10;
    short         s1 = 30000, s2 = 30000;
    char          c = 'D';

    printf("[1] int(-10) + unsigned(5)  = %d  (as unsigned: %u)\n", i + (int)u, i + u);
    printf("    (i + u) > 0 ?           = %d  <- -10 became 4294967286\n", (i + u) > 0);
    printf("[2] short 30000 + short 30000 = %d  (promoted to int, no overflow)\n", s1 + s2);
    short s3 = s1 + s2;
    printf("    stored back in short      = %d  (truncated)\n", s3);
    printf("[3] char 'D' + 1            = %d  (promoted to int, prints as %c)\n", c + 1, c + 1);

    /* ---- 2. integer division ---- */
    int units = 250;
    printf("\n[4] units / 100             = %d  (expected 2.5)\n", units / 100);
    printf("    units / 100.0           = %.2f\n", units / 100.0);
    printf("    2/3 * 150 units         = %d  (2/3 is 0 in integer division)\n", 2 / 3 * 150);
    printf("    150 * 2 / 3             = %d  (reordered: correct)\n", 150 * 2 / 3);

    /* ---- 3. explicit casting of large long ---- */
    long big = 5000000000L;               /* a district total in paise */
    int  cut = (int)big;
    printf("\n[5] (int)5000000000L        = %d  (high bits lost)\n", cut);
    long dist = 100000L * 65535L;
    printf("    100000*65535 as long    = %ld\n", dist);
    printf("    same product in int     = %d  (overflow)\n", (int)dist);

    /* ---- 4. rounding fractional charge to paise ---- */
    double charge = 162.505 * 100;       /* rupees -> paise */
    printf("\n[6] 162.505 rupees * 100    = %.10f\n", charge);
    printf("    truncate (int)          = %d\n", (int)charge);
    printf("    round half up (+0.5)    = %d\n", (int)(charge + 0.5));
    printf("    lround()                = %ld\n", lround(charge));
    printf("    0.1+0.2 == 0.3 ?        = %d  (0.1+0.2 = %.17f)\n", 0.1 + 0.2 == 0.3, 0.1 + 0.2);
    printf("    3 units * Rs 2.255      : %.4f -> %ld paise (round)  %d paise (trunc)\n",
           3 * 2.255, lround(3 * 2.255 * 100), (int)(3 * 2.255 * 100));
    return 0;
}
```

task3_alt.c — error growth with the number of bills n

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>

/* Rate for units 101-250 is Rs 2.255/unit (225.5 paise): fractional charge */
int main(void)
{
    int ns[] = {10, 100, 1000, 10000, 100000, 1000000};
    printf("%-9s %-15s %-15s %-15s\n", "bills n", "float(once)", "double(once)", "int paise/bill");
    printf("%-9s %-15s %-15s %-15s\n", "", "err (paise)", "err (paise)", "err (paise)");
    for (int k = 0; k < 6; k++) {
        int n = ns[k];
        unsigned seed = 12345;
        float  fsum = 0.0f;  double dsum = 0.0;
        long   exact10 = 0;           /* exact total in tenths of a paisa */
        long   isum = 0;              /* each bill rounded to whole paise */
        for (int i = 0; i < n; i++) {
            seed = seed * 1103515245u + 12345u;
```

```
            int units = 101 + (int)((seed >> 16) % 150);       /* 101..250 */
            long tenths = 5000L * 10 + (long)(units - 100) * 2255;
            exact10 += tenths;
            fsum += (float)tenths / 10.0f;
            dsum += (double)tenths / 10.0;
            isum += (tenths + 5) / 10;                          /* round half up */
        }
        double exact = exact10 / 10.0;
        printf("%-9d %-15.4f %-15.6f %-15.1f\n", n,
               (double)llround(fsum) - exact, (double)llround(dsum) - exact, (double)isum - exact);
    }
    /* time and space per value */
    long N = 50000000L; volatile double sink; clock_t t;
    float f = 0; t = clock(); for (long i = 0; i < N; i++) f += 1.5f;       double tf = (double)(clock()-
t)/CLOCKS_PER_SEC; sink = f;
    long  l = 0; t = clock(); for (long i = 0; i < N; i++) l += 15;          double tl = (double)(clock()-
t)/CLOCKS_PER_SEC; sink = (double)l;
    printf("\nTime for %ld additions: float %.3f s, long %.3f s\n", N, tf, tl);
    printf("Space per value: float %zu bytes, double %zu bytes, long %zu bytes\n",
           sizeof(float), sizeof(double), sizeof(long));
    (void)sink;
    return 0;
}
```

Output Screenshot

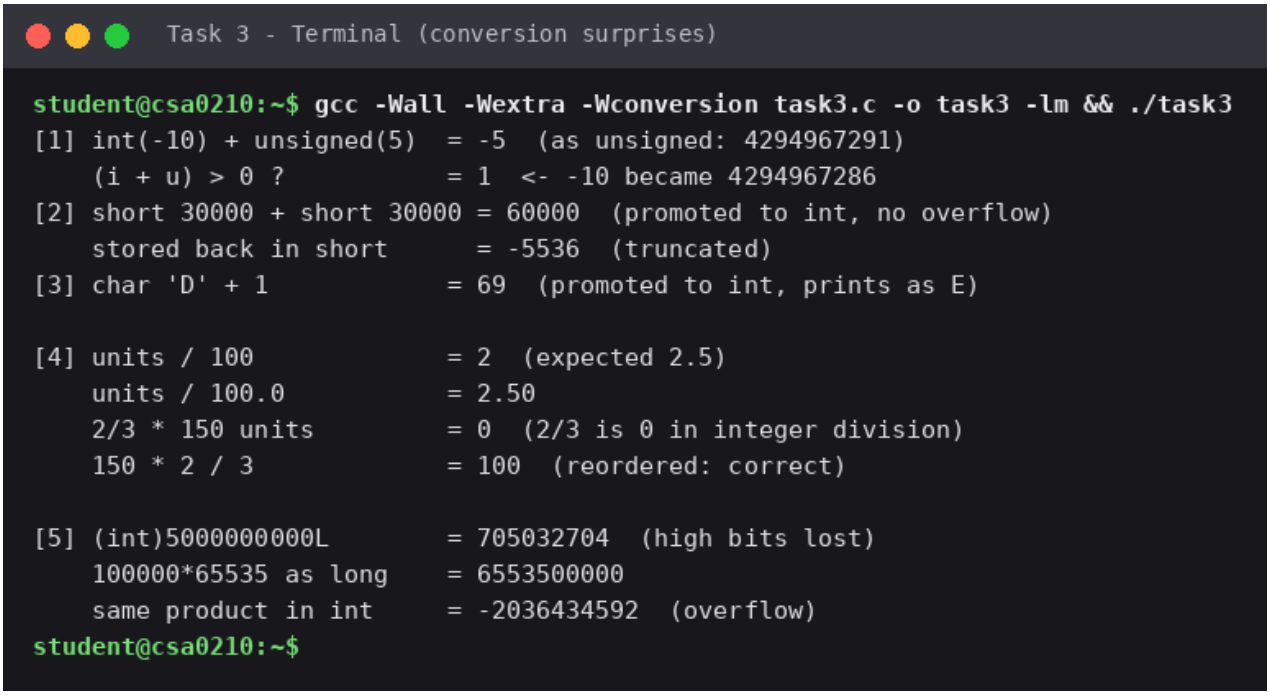


Figure 3.1 — Implicit conversion, integer division and casting results (expressions 1 to 5).

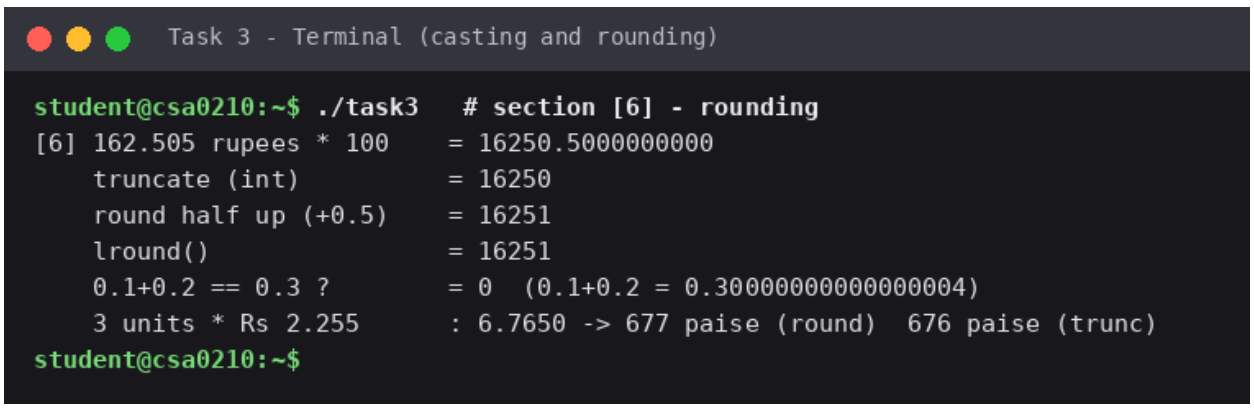


Figure 3.2 — Rounding a fractional charge to paise (expression 6).

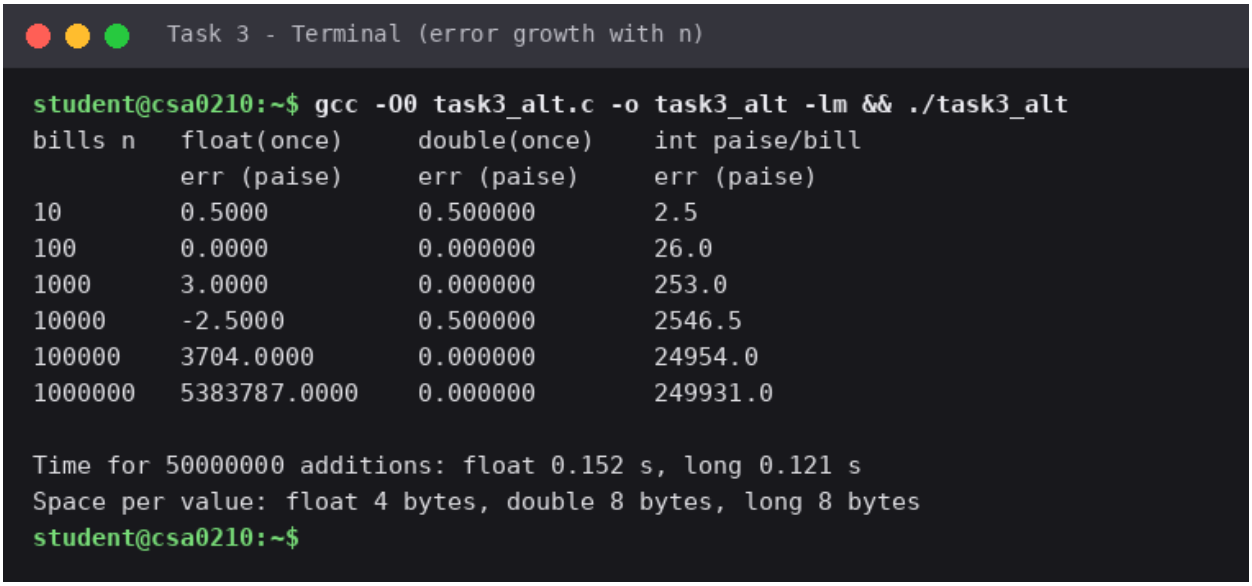


Figure 3.3 — Alternative approach: error of the total for float, double and integer paise as n grows.

Analysis / Discussion

Unexpected results and their explanation

Expression	Output	Explanation
`-10 + 5u` (int + unsigned), then `> 0`	`4294967291`; comparison is **true**	The usual arithmetic conversions turn the signed −10 into the unsigned 4,294,967,286, so the sum is a huge positive number. Printed as `%d` it shows −5, but as an unsigned value it is greater than 0. A bill check such as `if (credit + charge > 0)` would be wrong.
`units / 100` with units = 250	`2`, expected 2.5	Both operands are int, so the division truncates. `2 / 3 * 150` gives 0 because `2 / 3` is evaluated first and is 0; writing `150 * 2 / 3` (multiply first) gives the right 100, and `units / 100.0` gives 2.50.
`(int)5000000000L`	`705032704`	The 64-bit value does not fit in 32 bits. The cast keeps only the low 32 bits (5,000,000,000 − 2^{32}), which is a wrong, positive-looking amount. The district total 100000 × 65535 = 6,553,500,000 is correct as a `long` but becomes **−2036434592** when forced into an `int`.
`short 30000 + short 30000`	`60000`, stored in a short: `-5536`	The addition is done in `int` after promotion (no overflow), but storing 60,000 back in a 16-bit `short` keeps only the low 16 bits, so the value is 60,000 − 65,536.
`(int)(162.505 * 100)`	`16250` (truncation); `16251` with rounding	The fractional paisa is discarded by the cast, so the customer pays one paisa less on every such bill.

Consistent rounding rule. Every bill is rounded **once, to the nearest paisa with halves rounded up**, and the rounded bill is stored as an integer number of paise. With an integer calculation in tenths of a paisa this is `(tenths + 5) / 10`; with a `double` it is `lround(x)`. The rule is never "truncate in some places and round in others", and the total of all bills is the sum of the rounded bills.

Alternative Concept and Complexity Analysis — float for everything, rounded once at the end

In Figure 3.3 the rate for units 101 to 250 is 225.5 paise per unit, so many bills contain a half paisa. The table compares the total error (in paise) against the exact total for the three methods.

Bills n	float, round at end: error	double, round at end: error	integer paise, rounded per bill: error
10	0.5	0.5	2.5
100	0.0	0.0	26.0
1,000	3.0	0.0	253.0
10,000	−2.5	0.5	2,546.5
100,000	3,704	0.0	24,954
1,000,000	5,383,787 (about Rs 53,838)	0.0	249,931

Reading the table. The `float` error grows quickly and unpredictably: a `float` has only 24 bits of precision, so once the running total passes about 16.7 million paise (Rs 167,772) it can no longer hold a single paisa and additions of 17,000-paise bills are rounded away; at one million bills the total is wrong by more than Rs 53,000. `double` stays correct because its 53 bits hold the whole total of this size exactly. The integer method shows an error that grows **linearly** (about 0.25 paise per bill, 24,954 for 100,000 bills) because it rounds every bill, and half of the bills have a half paisa that is always rounded up. This is not a calculation fault: it is the deliberate and auditable rule above, and the integer total is exactly the sum of the bills actually issued. The `double`-rounded-once total is closer to the unrounded sum but it **does not equal the sum of the printed bills**, by the same 24,954 paise for 100,000 bills, which an accountant would find as a mismatch.

Aspect	float (once at end)	double (once at end)	long paise (per bill)
Space per value	4 bytes	8 bytes	8 bytes (`long`)
Time per addition (50 million additions)	O(1); 0.152 s in this run	O(1); about the same as float	O(1); 0.121 s in this run
Growth of error with n	Large and erratic; exceeds Rs 53,000 at $n = 10^6$	Negligible up to about 2^{53} paise	Linear, about 0.25 paise per bill, by a fixed rule
Total equals sum of printed bills	No	No	Yes

Time and space per value are about the same (differences between runs are within measurement noise), so the deciding factor is correctness: integer paise gives exact, repeatable totals that match the issued bills, while single-precision float is unsuitable for money at this scale.

Result

Eight places where types are combined were identified, five surprising results were demonstrated and explained (unsigned promotion, integer division, narrowing casts, short overflow, truncation of the fractional paisa), and a single rounding rule was adopted: round each bill once, half up, to whole paise. The alternative float calculation had an error of 5,383,787 paise at 1,000,000 bills against exact integer arithmetic, with the same O(1) time per addition.

TASK 4 — Trace the Compilation Process

Marks: 4 **Bloom's Level:** K3 – Apply

Task / Problem Statement

Follow the billing program through the stages of compilation: the output of each stage (`gcc -E` preprocessed, `gcc -S` assembly, `gcc -c` object file, and the linked executable), what the preprocessor removed or replaced, the difference between a compile-time error, a link-time error and a run-time error with one example of each, and what the warnings from `gcc -Wall -Wextra` told us and how they were resolved. As the alternative, split the program into several `.c` files (`input.c`, `tariff.c`, `report.c`) with a shared header, compile them separately and compare the build process, the compile time after a change to one file, and the space used by the object files.

Aim

To observe the output of each stage of the gcc pipeline, to classify errors correctly as compile-time, link-time or run-time, to fix compiler warnings, and to compare single-file and separate compilation.

Concept / Theory

Stage	Command	Output	What happens
1. Preprocessing	`gcc -E`	Text (`.i`)	Comments removed; `#include` replaced by the file contents; `#define` names replaced by their values; `#ifdef` lines kept or dropped.
2. Compilation	`gcc -S`	Assembly (`.s`)	Checks syntax and types; translates C into processor instructions.
3. Assembly	`gcc -c`	Object file (`.o`)	Assembly becomes machine code; calls to functions defined elsewhere (like `printf`) are left as **undefined symbols**.
4. Linking	`gcc a.o b.o -o prog`	Executable	The linker joins object files and libraries and resolves every undefined symbol.

Compile-time error: the compiler cannot translate the source (syntax error, undeclared name, type mismatch); no object file is produced. **Link-time error:** every file compiled, but the linker cannot find the definition of a function or variable (`undefined reference`). **Run-time error:** the program builds, but fails while running (division by zero, invalid memory access). **Warnings** are reported by the compiler for code that is legal but probably wrong; the program still builds, so they must be read and removed.

Procedure / Algorithm

1. Write the single-file billing program `bill_single.c` (22 lines).
2. Run `gcc -E` and examine the end of the output to see what the preprocessor did to the constants and comments.
3. Run `gcc -S`, then `gcc -c` (check with `nm`) and finally link and run the program.
4. Create one example of each error type: a missing semicolon, a missing object file at link time, and a division by zero at run time.
5. Compile a deliberately careless program with `-Wall -Wextra`, read each warning, and fix them.
6. Split the program into `input.c`, `tariff.c`, `report.c`, `main.c` with a shared header `bill.h` and a `Makefile`; touch one file and see what is rebuilt.
7. Measure the rebuild time and object-file sizes for a scaled-up version (3 files of 1,500 functions each) in the single-file and multi-file arrangements.

Program / Source Code

bill_single.c — program used for the stage trace

```
#include <stdio.h>

#define FIXED CHARGE PAISE 5000L   /* Rs 50 fixed charge */
#define RATE2 PAISE        225

/* compute bill in paise */
long compute_bill(int units)
{
    long p = FIXED CHARGE PAISE;
    if (units > 100) p += (long)(units - 100) * RATE2 PAISE;
    return p;
}

int main(void)
{
    int units = 150;
    long paise = compute bill(units);
    printf("Bill = Rs %ld.%02ld\n", paise / 100, paise % 100);
    return 0;
}
```

bill.h — shared header of the multi-file version

```
#ifndef BILL_H
#define BILL H
#define SLAB1 UNITS        100
#define SLAB2 UNITS        200
#define RATE2 PAISE        225
#define RATE3 PAISE        450
#define FIXED CHARGE PAISE 5000L
#define DEBUG_PRINT(x) /* debug off */

unsigned int read units(void);             /* input.c  */
long         compute bill(int units);      /* tariff.c */
void         print bill(unsigned u, long p);/* report.c */
#endif
```

input.c

```
#include <stdio.h>
#include "bill.h"

/* reads two meter readings and returns units consumed (rollover safe) */
unsigned int read units(void)
{
    unsigned short prev, curr;
    if (scanf("%hu %hu", &prev, &curr) != 2) return 0;
    return (unsigned short)(curr - prev);
}
```

tariff.c

```
#include "bill.h"

long compute bill(int units)
{
    long p = FIXED CHARGE PAISE;
    if (units > SLAB2 UNITS) p += (long)(units - SLAB2 UNITS) * RATE3 PAISE;
    if (units > SLAB1_UNITS)
        p += (long)((units > SLAB2_UNITS ? SLAB2_UNITS : units) - SLAB1_UNITS) * RATE2_PAISE;
    return p;
}
```

report.c

```
#include <stdio.h>
#include "bill.h"

void print bill(unsigned u, long p)
{
    printf("Units used   : %u\n", u);
    printf("Total payable: Rs %ld.%02ld\n", p / 100, p % 100);
}
```

main.c

```
#include <stdio.h>
#include "bill.h"

int main(void)
{
    unsigned int units = read units();     /* input.c  */
    long paise = compute bill((int)units); /* tariff.c */
    print bill(units, paise);              /* report.c */
    return 0;
}
```

Makefile

```
CC=gcc
CFLAGS=-Wall -Wextra
OBJS=main.o input.o tariff.o report.o
bill multi: $(OBJS)
    $(CC) $(OBJS) -o bill multi
%.o: %.c bill.h
    $(CC) $(CFLAGS) -c $< -o $@
clean:
    rm -f *.o bill_multi
```

err_compile.c — compile-time error (missing semicolon)

```
#include <stdio.h>
int main(void)
{
    int units = 150
    long paise = total paise(units);
    printf("%ld\n", paise);
    return 0;
}
```

err_run.c — run-time error (division by zero)

```
#include <stdio.h>
int main(void)
{
    int units = 0;
    long paise = 5000;
    printf("Cost per unit = %ld paise\n", paise / units);
    return 0;
}
```

warn.c — careless code that produces warnings

```
#include <stdio.h>

long compute_bill(int units, int category)
{
    long paise;
    int  unused total = 0;
    unsigned int limit = 200;
    if (units > limit)
        paise = 5000 + (units - 200) * 450L;
    printf("Bill = %d\n", paise);
}

int main(void)
{
    printf("%ld\n", compute bill(250, 1));
    return 0;
}
```

warn_fixed.c — the same code after fixing every warning

```
#include <stdio.h>

long compute bill(int units)
{
    long paise = 5000;                      /* initialised */
    const int limit = 200;                  /* same type as units */
    if (units > limit)
        paise += (long)(units - limit) * 450L;
    printf("Bill = %ld\n", paise);          /* %ld for long */
    return paise;                           /* value returned */
```

```
}

int main(void)
{
    printf("%ld\n", compute_bill(250));
    return 0;
}
```

Output Screenshot

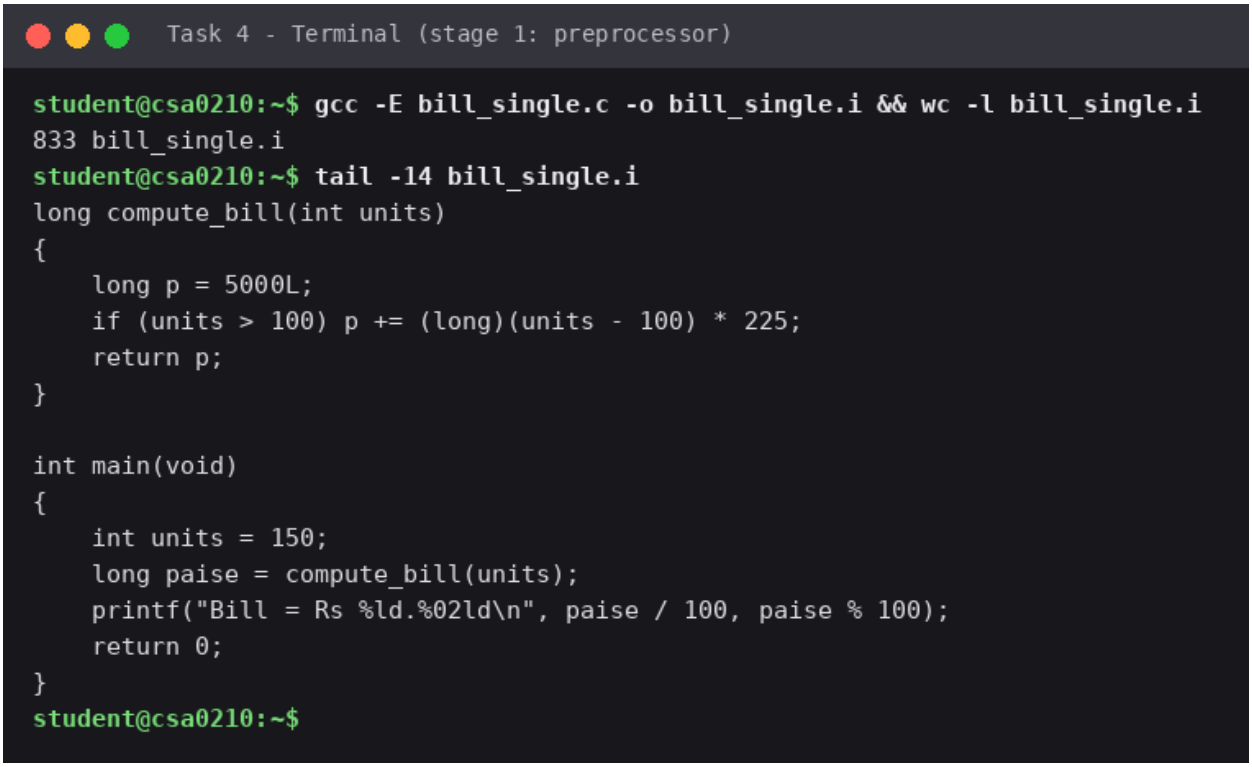


Figure 4.1 — Stage 1, gcc -E: the preprocessed file is 833 lines long; the end shows the macros replaced.

```
Task 4 - Terminal (stage 2: assembly)

student@csa0210:~$ gcc -S bill_single.c -o bill_single.s && sed -n 1,30p bill_single.s
    .file    "bill_single.c"
    .text
    .globl    compute_bill
    .type    compute_bill, @function
compute_bill:
.LFB0:
    endbr64
    pushq    %rbp
    movq    %rsp, %rbp
    movl    %edi, -20(%rbp)
    movq    $5000, -8(%rbp)
    cmpl    $100, -20(%rbp)
    jle    .L2
    movl    -20(%rbp), %eax
    subl    $100, %eax
    cltq
    imulq    $225, %rax, %rax
    addq    %rax, -8(%rbp)
.L2:
    movq    -8(%rbp), %rax
    popq    %rbp
    ret
.LFE0:
    .size    compute_bill, .-compute_bill
    .section    .rodata
.LC0:
    .string    "Bill = Rs %ld.%02ld\n"
    .text
    .globl    main
    .type    main, @function
student@csa0210:~$
```

Figure 4.2 — Stage 2, gcc -S: assembly for compute_bill (the constants appear as $5000 and $225).

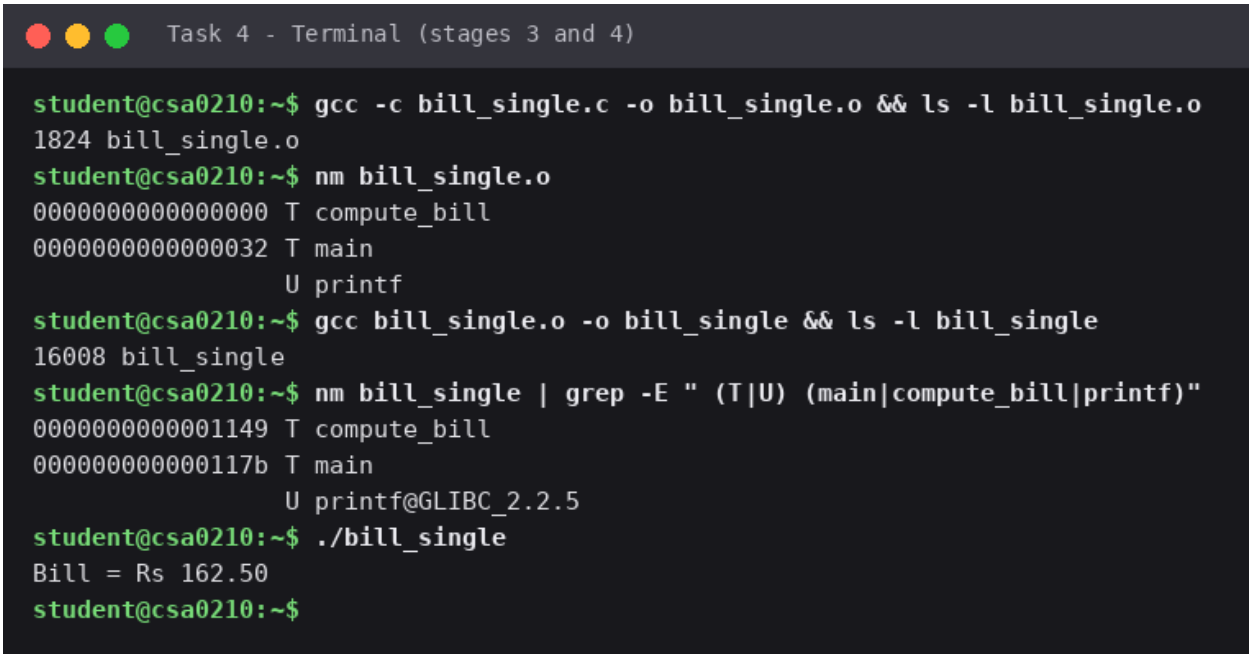


Figure 4.3 — Stages 3 and 4: object file (printf undefined), linked executable and program output.

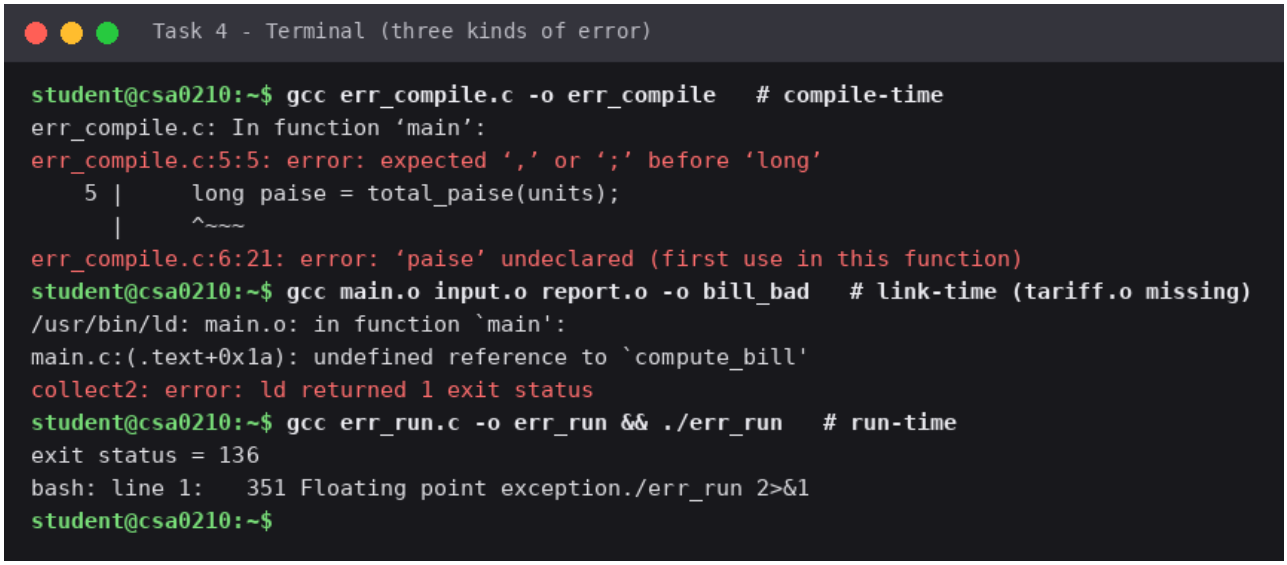


Figure 4.4 — One example each of a compile-time, a link-time and a run-time error.

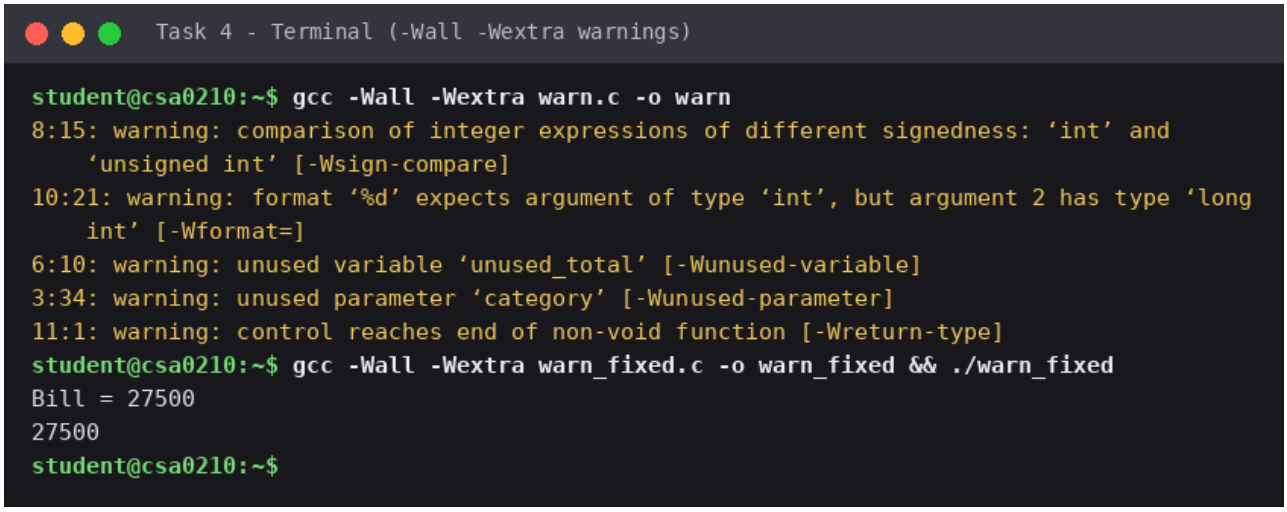


Figure 4.5 — Warnings from -Wall -Wextra, and the clean build after the fixes.



Figure 4.6 — Separate compilation with make: after changing tariff.c only that file is recompiled.

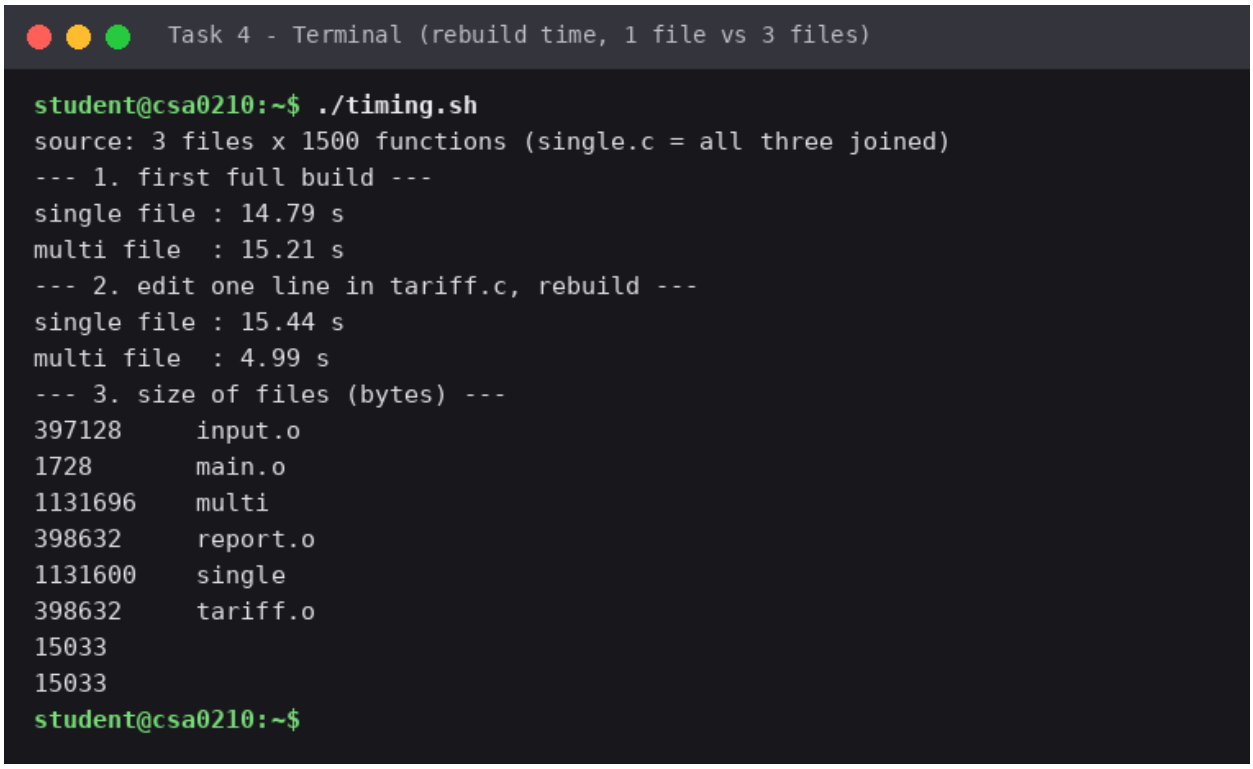


Figure 4.7 — Rebuild time and file sizes, single file against three files (scaled-up version).

Analysis / Discussion

Output of each stage

Stage 1 (Figure 4.1). The 22-line program grows to **833 lines** after `gcc -E`, because `#include <stdio.h>` was replaced by the whole text of the header (and the headers it includes). At the end of the file the two macros have been replaced by their values (`long p = 5000L;` and `* 225`), the comments `/* Rs 50 fixed charge */` and `/* compute bill in paise */` are gone, and the `#define` lines no longer exist. The compiler therefore never sees the names `FIXED_CHARGE_PAISE` or `RATE2_PAISE`.

Stage 2 (Figure 4.2). `gcc -S` gives about 110 lines of assembly. The function `compute_bill` contains `movq $5000, -8(%rbp)` (the fixed charge), `cmpl $100` (the slab test) and `imulq $225` (the rate): the constants are now plain numbers in the instructions. The format string of `printf` is placed in the read-only section as `.LC0`.

Stage 3 (Figure 4.3). `gcc -c` produced `bill_single.o` of 1,824 bytes. `nm` lists `compute_bill` and `main` as defined (`T`) and `printf` as **undefined** (`U`): the object file does not yet know where `printf` is, so it cannot be run.

Stage 4 (Figure 4.3). After linking the executable is 16,008 bytes, `printf` is now resolved to the C library (`U printf@GLIBC_2.2.5` — bound at load time to the shared library) and the program prints `Bill = Rs 162.50`.

Compile-time, link-time and run-time errors (Figure 4.4)

Type	Example from this work	Message	Fix
Compile-time	`int units = 150` without a semicolon in `err_compile.c`	`error: expected ',' or ';' before 'long'`; a second message (`'paise' undeclared`) is a consequence of the first	Add the `;`. Always fix the first error first.
Link-time	`main.o`, `input.o` and `report.o` linked without `tariff.o`	`undefined reference to 'compute_bill'` and `ld returned 1 exit status`	Add `tariff.o` to the link command (or let `make` do it).
Run-time	Cost per unit `paise / units` with `units = 0` in `err_run.c`	`Floating point exception`, exit status 136 (128 + signal 8, SIGFPE); compiled without any message	Test `units != 0` before dividing.

Warnings from -Wall -Wextra (Figure 4.5)

Warning (gcc)	Meaning	How it was resolved
`comparison of integer expressions of different signedness` (units > limit)	`int` compared with `unsigned int`; a negative `units` would become huge.	Declared `limit` as `const int`, the same type as `units`.
`format '%d' expects argument of type 'int', but argument 2 has type 'long int'`	Wrong specifier for a `long` (same fault as Task 2).	Used `%ld`.
`unused variable 'unused_total'`	Declared but never used.	Removed the variable.
`unused parameter 'category'`	Function accepts a value it never uses (the tariff is the same for all categories).	Removed the parameter from the function.
`control reaches end of non-void function`	`compute_bill` has no `return`; the caller would receive an undefined value.	Added `return paise;`.

Also in `warn.c` the variable `paise` was not set when `units <= 200`; the corrected version initialises it to the fixed charge. After the fixes `gcc -Wall -Wextra` reports no warnings and the program prints the correct value (27,500 for 250 units in this simplified function).

Alternative Concept and Complexity Analysis — separate compilation of several .c files

The program is split so that `input.c` reads the meter readings, `tariff.c` computes the bill and `report.c` prints it; the shared declarations are in `bill.h`. Each `.c` file is compiled to its own `.o` file, and the linker joins them. `make` (Figure 4.6) rebuilds only what depends on a changed file: after `touch tariff.c` only two commands run (compile `tariff.c`, relink) instead of five; after a change to `bill.h` all four files are recompiled, because all of them include it.

The billing files are so small that the compile time is a few milliseconds either way, so a scaled-up version was measured (Figure 4.7): three files with 1,500 functions each, and the same code joined into one file.

Measurement	One file	Three files + main	Comment
First full build	14.79 s	15.21 s	About the same: the same total code is compiled.
Rebuild after changing one line in `tariff.c`	15.44 s	4.99 s	Multi-file is about 3 times faster.
Final executable	1,131,600 bytes	1,131,696 bytes	Almost identical.
Object files kept on disk	none needed	397,128 + 398,632 + 398,632 + 1,728 = 1,196,120 bytes	About one extra copy of the code.

Time. With one file, any change costs O(S) where S is the size of the whole program, because everything is recompiled. With separate files the cost after a change is O(S_i + link), where S_i is the size of the changed file (plus the other files only when the shared header changes), which is about one third of the work here. The first build is not faster, since the total amount of code is the same, and each file also re-reads the headers. **Space.** The separate object files add roughly one more copy of the code on disk (about 1.2 MB here, 6,208 bytes for the real billing files) but the final executable is the same size, so the extra space is only paid on disk during development. For large programs separate compilation is the standard method because the rebuild time stays proportional to the change, and the files can be written and tested separately.

Result

The billing program was traced through preprocessing (22 → 833 lines, constants replaced, comments removed), compilation to assembly, assembly to an object file with undefined symbol `printf`, and linking to a working executable. Each of the three error types was produced and classified (missing semicolon, missing object file, division by zero), five gcc warnings were explained and removed, and separate compilation was shown to rebuild after a one-file change in 4.99 s against 15.44 s for a single file, using about 1.2 MB more disk space for object files.

Test Cases / Validation Table

Different inputs, expected results, actual results and status for the four tasks.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	Task 1: normal build, 0 units	Rs 50.00	Rs 50.00	Pass
2	Task 1: normal build, 150 units	Rs 162.50	Rs 162.50	Pass
3	Task 1: normal build, 350 units	Rs 950.00	Rs 950.00	Pass
4	Task 1: `-DDEBUG` build, 350 units	Same bill plus [DEBUG] lines	Rs 950.00 with 3 [DEBUG] lines	Pass
5	Task 1: header included twice, no guard	Compiles	`conflicting types for 'Consumer'`	Defect shown (fixed by guard)
6	Task 1: header included twice, with guard	Compiles	Compiles, no errors	Pass
7	Task 2: readings 1200, 1530	330 units, Rs 860.00	330 units, Rs 860.00	Pass
8	Task 2: readings 65400, 120 (rollover)	256 units, Rs 527.00	256 units, Rs 527.00	Pass
9	Task 2: input `4071 D abc 1530`	Rejected	"only 2 of 4 items were read", exit 1	Pass
10	Task 2: `%d` printing 5000000000 (long)	5000000000	705032704	Defect shown (use `%ld`)
11	Task 2: fgets+strtol with abc, 12x, 70000, 1530	First three rejected, 1530 accepted	As expected, 4 attempts	Pass
12	Task 3: 250 / 100	2.5	2	Defect shown (integer division)
13	Task 3: (int)5000000000L	5000000000	705032704	Defect shown (narrowing cast)
14	Task 3: -10 + 5u > 0	false (−5)	true (4294967291)	Defect shown (unsigned conversion)
15	Task 3: rounding 16250.5 paise to nearest	16251	16251 (lround and +0.5); 16250 by truncation	Pass (rule adopted)
16	Task 3: 1,000,000 bills, float total	Exact total	Error 5,383,787 paise	Fails — integer paise preferred
17	Task 4: gcc -E of 22-line program	Macros replaced, comments removed	833 lines; 5000L and 225 substituted	Pass
18	Task 4: link without tariff.o	Link error	`undefined reference to 'compute_bill'`	Error classified (link-time)
19	Task 4: warn_fixed.c with -Wall -Wextra	0 warnings	0 warnings	Pass
20	Task 4: touch tariff.c, run make	Only tariff.c recompiled + relink	2 commands run	Pass

Final Submission Checklist

- Student and course details completed (check the register number)
- Task statement, aim, theory, and algorithm included
- Program code is readable and runs successfully

- Output screenshot included
- Analysis and result included
- Test cases completed
- GitHub repository link added if required